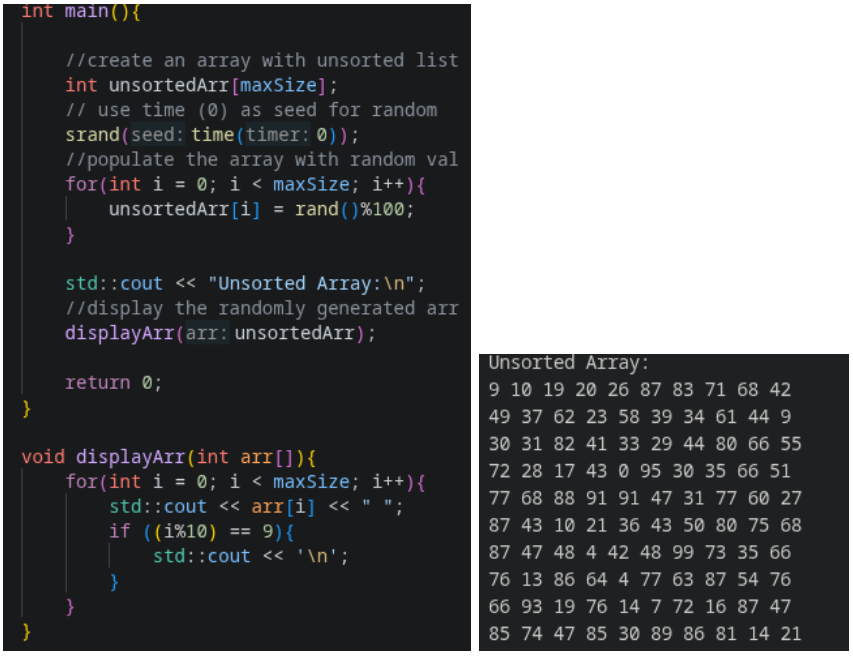
COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	9/4/26	**DATE SUBMITTED:**	9/8/26
NAME:	Condino, Niel Vincent B.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 7.1 **Sorting Algorithms Pt. 1**

1. PROCEDURE OUTPUT

Code + Console Screenshot	
Observations	This snippet of the code populates an array with random numbers using the rand() function. This loop runs n times based on the desired size of the array. The displayArr() function also runs O(n) times as it prints out every 10 elements of the array in a row. The combined runtime complexity of the code is O(n) in its worst and best case scenarios. The output is an array populated with pseudorandom numbers generated using the seed of time(0).

```
int main(){

    //create an array with unsorted list
    int unsortedArr[maxSize];
    // use time (0) as seed for random
    srand(seed: time(timer: 0));
    //populate the array with random val
    for(int i = 0; i < maxSize; i++){
        unsortedArr[i] = rand()%100;
    }

    std::cout << "Unsorted Array:\n";
    //display the randomly generated arr
    displayArr(arr: unsortedArr);

    return 0;
}

void displayArr(int arr[]){
    for(int i = 0; i < maxSize; i++){
        std::cout << arr[i] << " ";
        if ((i%10) == 9){
            std::cout << '\n';
        }
    }
}
```

```
Unsorted Array:
9 10 19 20 26 87 83 71 68 42
49 37 62 23 58 39 34 61 44 9
30 31 82 41 33 29 44 80 66 55
72 28 17 43 0 95 30 35 66 51
77 68 88 91 91 47 31 77 60 27
87 43 10 21 36 43 50 80 75 68
87 47 48 4 42 48 99 73 35 66
76 13 86 64 4 77 63 87 54 76
66 93 19 76 14 7 72 16 87 47
85 74 47 85 30 89 86 81 14 21
```

Table 7-1. Array of Values for Sort Algorithm Testing

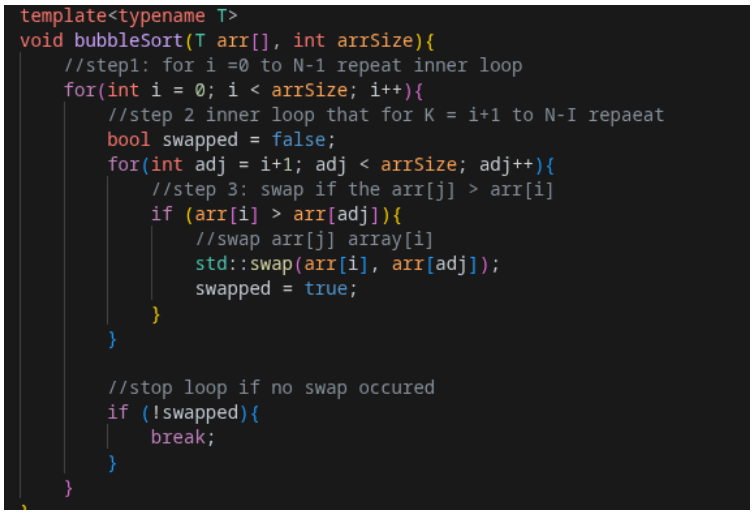
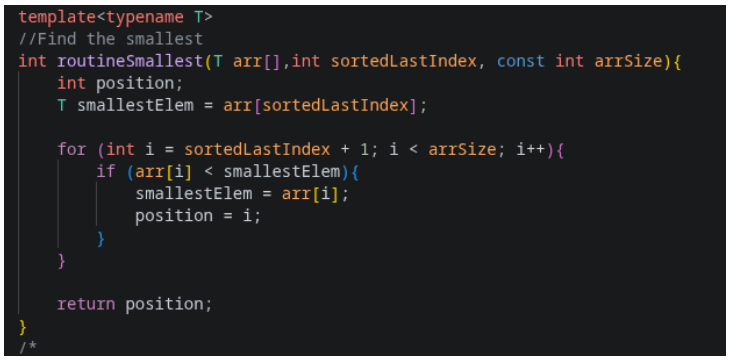
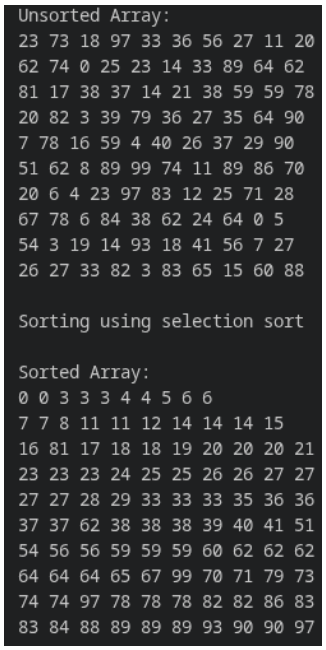
Code + Console Screenshot	 Unsorted Array: 31 39 14 97 40 9 56 21 77 34 56 59 94 62 50 71 99 31 65 67 75 43 36 34 65 31 82 89 62 33 21 93 24 35 90 16 96 99 37 73 85 46 85 31 8 87 3 7 70 20 75 46 15 63 32 81 95 66 22 57 99 43 2 76 78 44 92 27 43 30 52 28 76 37 12 36 77 15 43 47 87 70 45 54 86 77 35 81 44 57 90 95 52 44 71 83 88 64 10 32 Sorting using bubble sort Sorted Array: 31 39 14 97 40 9 56 21 77 34 56 59 94 62 50 71 99 31 65 67 75 43 36 34 65 31 82 89 62 33 21 93 24 35 90 16 96 99 37 73 85 46 85 31 8 87 3 7 70 20 75 46 15 63 32 81 95 66 22 57 99 43 2 76 78 44 92 27 43 30 52 28 76 37 12 36 77 15 43 47 87 70 45 54 86 77 35 81 44 57 90 95 52 44 71 83 88 64 10 32
Observations	This algorithm swaps every element of the array until the largest value of the array “bubbles” to the end of the array. Inspecting the code of this algorithm, it runs in $O(n^2)$. This runtime analysis is based on the nested loops inside the program, that has both a runtime cost of O(n).

Table 7-2. Bubble Sort Technique

Code + Console Screenshot	  
Observations	This algorithm divides the array into a sorted and unsorted section. It then scans the array for the smallest number in the unsorted section, then appends it to the sorted section. In this algorithm, it has a nested loop that both run n times. Hence, the final runtime complexity of this algorithm is $O(n^2)$

```
template<typename T>
//Find the smallest
int routineSmallest(T arr[],int sortedLastIndex, const int arrSize){
    int position;
    T smallestElem = arr[sortedLastIndex];

    for (int i = sortedLastIndex + 1; i < arrSize; i++){
        if (arr[i] < smallestElem){
            smallestElem = arr[i];
            position = i;
        }
    }

    return position;
}
/*
```

```
template<typename T>
void selectionSort(T arr[],int arrSize){
    int smallestElemIndex,pass;
    T temp;

    for (int i = 0; i < arrSize; i++){
        //get the smallest element
        smallestElemIndex = routineSmallest(arr, sortedLastIndex: i, arrS
        //swap the smallest element with the current index
        temp = arr[i];
        arr[i] = arr[smallestElemIndex];
        arr[smallestElemIndex] = temp;
        pass++;
    }
}
```

```
Unsorted Array:
23 73 18 97 33 36 56 27 11 20
62 74 0 25 23 14 33 89 64 62
81 17 38 37 14 21 38 59 59 78
20 82 3 39 79 36 27 35 64 90
7 78 16 59 4 40 26 37 29 90
51 62 8 89 99 74 11 89 86 70
20 6 4 23 97 83 12 25 71 28
67 78 6 84 38 62 24 64 0 5
54 3 19 14 93 18 41 56 7 27
26 27 33 82 3 83 65 15 60 88

Sorting using selection sort

Sorted Array:
0 0 3 3 3 4 4 5 6 6
7 7 8 11 11 12 14 14 14 15
16 81 17 18 18 19 20 20 20 21
23 23 23 24 25 25 26 26 27 27
27 27 28 29 33 33 33 35 36 36
37 37 62 38 38 38 39 40 41 51
54 56 56 59 59 59 60 62 62 62
64 64 64 65 67 99 70 71 79 73
74 74 97 78 78 78 82 82 86 83
83 84 88 89 89 89 93 90 90 97
```

Table 7-3. Selection Sort Algorithm

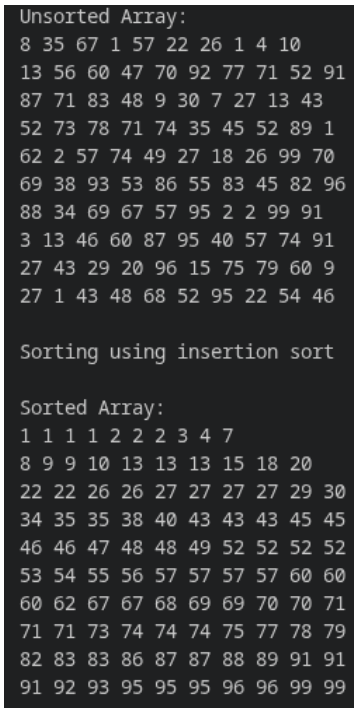
Code + Console Screenshot	 
Observations	Like selection sort, it divides the array into sorted and unsorted halves. However, this algorithm continually compares and shifts the current element in the sorted half until it finds its desired position. This has a nested loop running both n times. The runtime complexity of this whole algorithm is $O(n^2)$

Table 7-4. Insertion Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

```
START
   CREATE integer array voteCount of size 5 INITIALIZED TO 0

   CALL insertionSort(voteArray, arrSize)

   FOR i FROM 0 TO arrSize - 1 DO
      INCREMENT voteCount[voteArray[i] - 1] BY 1
   END FOR

   INITIALIZE maxVotes and winner = 0

   FOR i FROM 0 TO 4 DO
      IF voteCount[i] > maxVotes THEN
         maxVotes = voteCount[i]
         winner = i
      END IF
      PRINT (i + 1) + "s counted: " + voteCount[i]
   END FOR

   RETURN winner
END
```

Figure 1: Pseudocode of Algorithm

The pseudocode above lays the blueprint for the code in the next figure. In this pseudocode, I planned to use different algorithms such as insertion sort to sort the array. After the array is sorted, A linear search is used to find the maximum number of votes using a frequency array. After using the frequency array to search for the maximum value, the algorithm then returns the index of the maximum value as the index of the winner; this will be used later for accessing the array of candidate names to print out the winner.

```cpp
int getMajorityVote(int voteArray[],const int& arrSize){
    int voteCount[5] = {[0]=0};
    insertionSort(arr: voteArray, arrSize);
    //count occurances
    for (int i = 0; i < arrSize; i++){
        voteCount[voteArray[i] - 1]++;
    }

    //find max
    int max = 0;
    int winner;

    std::cout << '\n';
    for (int i = 0; i < 5; i++){
        if (voteCount[i] > max){
            max = voteCount[i];
            winner = i;
        }
        std::cout << i+1 <<"s counted: " << voteCount[i] << '\n';
    }
    return winner;
}
```

Figure 2: Screenshot of Algorithm Code

This code applies the pseudocode from above. Inspecting the code, there are things to keep in mind to determine its runtime complexity. The insertion sort function has a worst-case $O(n^2)$. The next is the linear search, which has an O(1) runtime instead of O (n) due to the determined 5 candidates. To further make the code adaptable to different inputs, a dynamic array can be used to set the size of the voteCount array. The sorting algorithm is also pretty unnecessary for counting votes. Overall, this algorithm runs at $O(n^2)$ time complexity.

I used insertion sort instead of others because of its advantages compared to the other algorithms. All of the algorithms have $O(n^2)$ runtime. With the optimized version of bubble sort, both it and insertion sort have O(n) runtime complexity. I still used insertion sort because of the lower number of operations it does per element. Bubble sort requires repeated pair-swapping, while insertion sort simply shifts elements to the right and places the key in its final position, drastically reducing write operations.

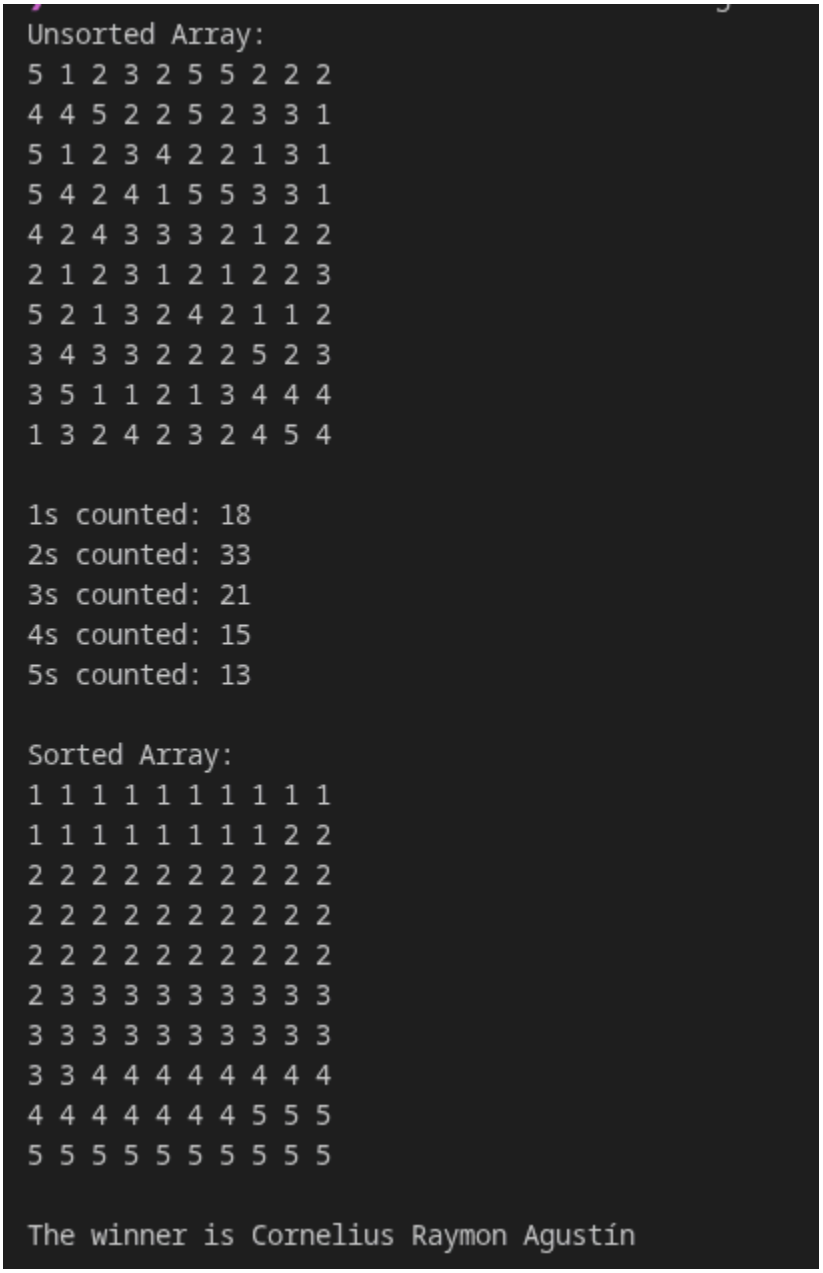
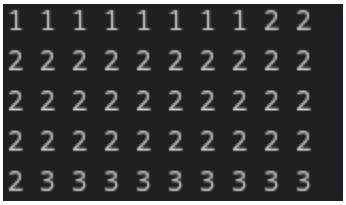
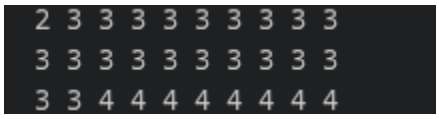
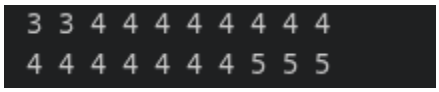


Figure 3: Output testing

The figure above shows the output of the code. It shows the unsorted array, the count of each element, and the sorted array. The last line then shows the winner of the votes. The output is the expected output from the given code. From the frequency count, candidate number 3 is declared the winner. The index for the array of {"Bo Dalton Capistrano", "Cornelius Raymon Agustín", Deja Jayla Bañaga", "Lalla Brielle Yabut", Franklin Relano Castro"} is accessed to print the correct winner.

Output Console Showing Sorted Array	Manual Count	Count Result of Algorithm
	18	18
	33	33
	21	21
	15	15
	13	13

Question: Was your developed vote counting algorithm effective? Why or why not?

The algorithm developed works effectively at counting the total number of votes for this input size. For this small number of votes, it runs in an acceptable time. This counting algorithm runs at O(n) time complexity. As the input increases, the runtime increases linearly. If the input size increases to, say, thousands or millions, I think I will need to optimize this algorithm further.

Conclusion:

In this activity, I am able to learn and apply my knowledge about the different sorting algorithms discussed. The visualizations of sorting algorithms helped me further understand these algorithms. Basically, each of them sorts an array, but the way of doing those are different, causing them to have different runtimes. All of them have a worst-case $O(n^2)$, but some algorithms outperform others and are preferred by programmers. In the supplementary, I was able to apply my past knowledge of creating algorithms and using linear search to check for the max frequency inside the frequency table created. I view this lesson as interesting, as some algorithms for sorting are much faster than others. These algorithms are just the beginning, and there are other much faster sorting algorithms.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

COMPUTER ENGINEERING

QUEZON CITY

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.